
KAN for Language Models: Empirical Interpretability and the MLPEdge Variant

Felippe Alves

Telus Digital Research Hub – CIAAM
Institute of Mathematics and Statistics
University of São Paulo
felippe.pereira@usp.br

Renato Vicente

Telus Digital Research Hub – CIAAM
Institute of Mathematics and Statistics
University of São Paulo
rvicente@usp.br

Abstract

Kolmogorov-Arnold Networks (KANs) replace fixed activation functions with learnable univariate functions on every edge, yielding ~ 147 K visualizable 1D curves per language model layer and a strong interpretability promise. We present the first empirical study of KAN edge interpretability at language model scale, measuring nonlinearity, activity, roughness, and functional principal components across all edges of KAN-based transformers trained on a 60 K-sample instruction-following corpus. Our study finds that **87.8%** of edges are meaningfully nonlinear ($\text{NLS} > 0.1$) and only 0.4% are dead, providing the first data-driven assessment of the KAN interpretability claim in a language model. A Mann-Whitney U test on all 884,736 edges yields $p < 10^{-300}$ for $\text{KAN} > \text{MLP}$, though this p -value should be read descriptively rather than inferentially: edges within the same layer share the same B-spline grid and basis, violating the independence assumption and inflating the effective sample size. The quantitative gap is modest (median NLS 0.281 vs. 0.268), and the per-layer distributions are consistent, suggesting the directional result is robust.

Motivated by the observation that the KAN topology—additive decomposition over edges—may matter more than the specific B-spline basis within the KAN-family design space, we introduce MLPEDGE: a drop-in replacement for the spline edge function with a tiny per-edge MLP ($\mathbb{R} \rightarrow \mathbb{R}$). We also run a follow-up function-basis screen over rational, polynomial, Gaussian/RBF, compact-support, and ReLU-power edge bases. On GUPPYLM, the strongest new basis row is a grouped Chebyshev degree-3 FFN, which reaches $0.2770_{\pm 0.0003}$ validation loss over three seeds, statistically indistinguishable from the same-protocol MLP baseline ($0.2770_{\pm 0.0008}$) but slower in the same-device inference sweep (550 ms vs. 309 ms). The corrected canonical GR-KAN row remains close to MLP ($0.2780_{\pm 0.0005}$), and a square-denominator rational ablation is similar ($0.2780_{\pm 0.0009}$). MLPEDGE remains the strongest *edge-function* replacement among the B-spline and MLPEDGE rows ($0.2854_{\pm 0.0005}$ vs. $0.2883_{\pm 0.0005}$ for the standard-attention B-spline baseline), while the Gaussian/RBF and compact-support bases fail the local quality screen. Thus no tested KAN-family FFN dominates the standard MLP FFN on both quality and latency at this scale. A GPT-2 scale experiment on Wikitext-103 (124M parameters, 20 000 steps) confirms the scale caveat: the MLP baseline reaches perplexity 16.58 versus 20.96 for the parameter-matched MLPEDGE ($h = 8$), widening rather than reversing the GuppyLM gap. For GR-KAN, the analogous scale test is also negative. Training from scratch at identical scale (286M parameters, ClimbMix corpus, H100 80GB HBM3) under the canonical Safe Padé rational definition, we sweep the group count: the best corrected configuration, $g = 4$, reaches $\text{val_bpb} = 0.890$ versus 0.8464 for the MLP baseline (gap +0.044, $\sim 5.2\%$), while $g = 8$ is worse (0.9030, +0.057,

$\sim 6.7\%$)—a higher group count gives no benefit at this scale. On DCLM CORE (22 ICL tasks, centered accuracy), $g = 4$ scores 0.128 and $g = 8$ scores 0.116, both below the MLP baseline of 0.154 ($\Delta = -0.026$ and -0.038); MLP leads on 18 of 22 tasks even against the stronger $g = 4$ variant. These results are notably weaker than preliminary estimates reported in an earlier version of this work, which were inflated by an implementation error in the rational activation denominator (per-coefficient absolute value instead of the canonical absolute-value-after-sum). We document this correction transparently. The corrected gap is wider than the $\sim 1.2\%$ preliminary estimate, so we treat this as a cautionary scale result: rational activations remain less damaging than the per-edge MLPEDGE factorization at this scale, but the current evidence does not support a GR-KAN scaling advantage over MLP at any tested group count. All code and checkpoints are released at <https://github.com/HCAI-USP/kan-guppylm>.

1 Introduction

Kolmogorov-Arnold Networks [Liu et al., 2025] replace the fixed activation functions of standard MLPs with learnable univariate functions on each edge, grounded in the Kolmogorov-Arnold representation theorem. The appeal for interpretability is direct: every edge $f_{i,j}(x)$ is a visualizable 1D function that can, in principle, be symbolically regressed into a closed-form expression. Subsequent work adapts KANs to transformers [Yang and Wang, 2025] and improves their computational efficiency [Blealtan, 2024], but the interpretability promise remains largely theoretical.

Two empirical gaps. First, no study has measured what KAN edge functions actually learn in a language model: are they nonlinear, active, and structurally diverse, or do most edges collapse to near-linear functions? Second, no work has asked whether the B-spline basis is necessary for quality, or whether the KAN topology alone is responsible for the gains over MLP transformers.

This paper. We address both gaps using GUPPYLM [arman bd, 2024] as a controlled testbed—a 60 K-sample fish-personality chatbot dataset small enough for exhaustive edge-level analysis (~ 147 K functions per FFN layer at full resolution) yet rich enough to produce coherent chat behavior. Our contributions are:

1. **Interpretability study.** We reconstruct and statistically characterize all learned 1D edge functions in KAN-based transformers, reporting nonlinearity scores (NLS), activity, roughness, functional PCA archetypes, and a direct comparison with MLP effective edge functions via Mann-Whitney U test (Section 6).
2. **MLPEDGE.** We propose replacing B-spline edge functions with tiny per-edge MLPs ($\mathbb{R} \rightarrow \mathbb{R}$), preserving the KAN topology while eliminating the grid mechanism. Within the B-spline/MLPEDGE subset, MLPEDGE improves over the B-spline baselines on validation loss and heuristic prompt quality; a same-protocol MLP baseline and the new Chebyshev basis screen are stronger on the core training objective (Section 5).
3. **Function-basis screen.** We test rational denominator variants, Chebyshev/Legendre polynomial bases, Gaussian/RBF bases, compact-support bases, and ReLU-power bases. A Chebyshev degree-3 grouped basis matches the MLP validation-loss mean but is slower; Gaussian/RBF and hat-style localized bases do not pass the GuppyLM quality screen (Section 5.2).

2 Background

2.1 KAN and the Kolmogorov-Arnold Theorem

The Kolmogorov-Arnold representation theorem states that every continuous multivariate function $f : [0, 1]^n \rightarrow \mathbb{R}$ can be written as:

$$f(\mathbf{x}) = \sum_{q=0}^{2n} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right) \quad (1)$$

where $\phi_{q,p}$ and Φ_q are continuous univariate functions. Liu et al. [2025] use this as motivation for replacing MLP weight matrices with learned function arrays, noting that the exact theorem applies to two-layer networks but the architecture generalizes to arbitrary depth.

2.2 KANLinear (EfficientKAN)

Blealtan [2024] implement KAN layers efficiently using B-spline basis functions:

$$\text{forward}(\mathbf{x}) = \mathbf{x} \odot \text{SiLU}(\mathbf{x}) \mathbf{W}_{\text{base}}^{\top} + \underbrace{\text{einsum}(\text{"bik,oik} \rightarrow \text{bo"}, B(\mathbf{x}), \mathbf{W}_{\text{spline}})}_{\text{spline path}} \quad (2)$$

where $B(\mathbf{x}) \in \mathbb{R}^{B \times d_{\text{in}} \times n_{\text{basis}}}$ is the B-spline basis evaluated via the Cox-de Boor recursion, $\mathbf{W}_{\text{spline}} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}} \times n_{\text{basis}}}$, and $n_{\text{basis}} = \text{grid_size} + \text{spline_order}$. Grid knots adapt to the activation distribution after a warm-up phase (*grid update*).

2.3 KAT Attention

Yang and Wang [2025] replace the linear Q and K projections with KANLinear feature maps φ , turning the dot-product similarity into a learned kernel:

$$K(\mathbf{q}, \mathbf{k}) = \varphi(\mathbf{q}) \cdot \varphi(\mathbf{k}) \quad (3)$$

3 Models

All models share the same base transformer: 6 layers, $d_{\text{model}} = 384$, 6 attention heads, trained on the GUPPYLM corpus with prompt-response masking (loss on assistant tokens only).

MLP-4x-GELU baseline. For the controlled small-scale comparison in Section 5, the standard transformer FFN is $\text{Linear}(384 \rightarrow 1536) \rightarrow \text{GELU} \rightarrow \text{Linear}(1536 \rightarrow 384)$.

KANPREY. For the controlled small-scale comparison in Section 5, KANPREY replaces each FFN with a single KANLINEAR layer ($384 \rightarrow 384$, $\text{grid}=2$, $\text{order}=3$), producing $384 \times 384 = 147,456$ learnable 1D edge functions per layer ($6 \times 147,456 = 884,736$ total across all FFN layers). It uses standard dot-product attention (not KAT).

KATPREY. Extends KANPREY by adding KANLinear feature maps to Q and K projections in every attention head (KAT attention [Yang and Wang, 2025]).

MLPEDGE (proposed). Uses standard dot-product attention (same as KANPREY, *not* KAT). Replaces the B-spline edge function $f_{i,j}(x)$ with a tiny per-edge MLP:

$$\mathbf{H} = \sigma(\text{einsum}(\text{"bi,ih} \rightarrow \text{bih"}, \mathbf{x}, \mathbf{W}_1) + \mathbf{b}_1) \in \mathbb{R}^{B \times d_{\text{in}} \times h} \quad (4)$$

$$\text{out} = \text{einsum}(\text{"bih,oih} \rightarrow \text{bo"}, \mathbf{H}, \mathbf{W}_2) + \mathbf{b}_{\text{out}} \in \mathbb{R}^{B \times d_{\text{out}}} \quad (5)$$

where h is the per-edge hidden size (default $h = 5$; the B-spline baseline uses $n_{\text{basis}} = 8$, so $h = 8$ is parameter-matched while $h = 5$ uses fewer parameters per edge). The einsum structure is identical to KANLINEAR, so $f_{i,j}(x)$ is still a visualizable 1D scalar function. The `update_grid()` call is a no-op: no grid mechanism is needed.

BASISKAN screen. For the function-basis experiments, we keep the standard transformer block and replace the FFN nonlinearity with grouped scalar bases followed by the same linear mixing dimensions as the MLP FFN. The screened bases include Chebyshev and Legendre recurrences, Gaussian RBFs, inverse-quadratic RBFs, Wendland compact-support polynomials, triangular and quadratic hats, and symmetric ReLU-power functions. The RBF and hat variants normalize pre-basis activations with `atanh`, use 8 centers uniformly spaced on $[-1, 1]$, and set the Gaussian/RBF width to `width_scale/7`.

Figure 1 illustrates the three primary FFN variants.

4 Experimental Setup

Dataset. arman-bd/guppylm-60k-generic [arman bd, 2024]: 60 K instruction-response pairs in ChatML format (fish-personality chatbot). BPE tokenizer trained on the same data, vocab size 2 393.

Training. Main comparison rows use 8 000 steps, batch size 32, cosine LR schedule (peak 3×10^{-4} , min 3×10^{-5}), AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$, $\lambda = 0.1$), and gradient clipping 1.0. Prompt-response masking sets targets for the user prompt portion to -100 (ignored in cross-entropy loss). Grid update for KAN/KAT models occurs at step 500. The function-basis screen uses one seed-42, 2 000-step pilot for each candidate; promoted candidates are rerun for 8 000 steps over seeds $\{42, 43, 44\}$.

Evaluation. (a) Validation cross-entropy on a held-out split; (b) keyword-match accuracy on 16 fixed prompts with human-defined keyword sets (pass/fail per prompt); (c) mean inference latency over 16 prompts $\times$ 64 new tokens.

Why GUPPYLM as testbed. At $d_{\text{model}} = 384$ and 6 layers, each FFN KANLINEAR layer has $384 \times 384 = 147,456$ edges—small enough to reconstruct and characterize every 1D function exhaustively, which is infeasible at GPT-2 scale ($d_{\text{model}} = 768$, $4\times$ more edges per layer, $4\times$ more parameters, orders-of-magnitude more compute per study). The single-domain corpus provides coherent semantic categories for token-conditioned interpretability analysis.

5 Results

5.1 Main comparison

Table 1: Main GUPPYLM results after protocol cleanup and the function-basis screen. Validation loss is mean $\pm$ std over 3 seeds for every 8 000-step row. Eval (16) and latency are measured on seed-42 checkpoints under same-device MPS sweeps with 16 prompts and 64 generated tokens; they are heuristic behavioral probes rather than replacements for validation loss. KATPREY changes both attention and FFN, so it should not be interpreted as an FFN-only ablation. The canonical GR-KAN row uses the corrected Safe Padé denominator.

Model	Attn.	FFN	Val loss	Eval	Lat.	Params
MLP-4x-GELU	Dot	MLP	0.2770 ± 0.0008	11/16	309 ms	11.62M
BASISKAN Cheb $d = 3$, $g = 8$	Dot	Cheb	0.2770 ± 0.0003	11/16	550 ms	11.62M
GR-KAN canonical	Dot	Rational	0.2780 ± 0.0005	14/16	459 ms	11.62M
GR-KAN square denom.	Dot	Rational	0.2780 ± 0.0009	15/16	382 ms	11.62M
KANPREY grid=2	Dot	Spline	0.2883 ± 0.0005	8/16	613 ms	9.83M
KATPREY	KAT	Spline	0.2867 ± 0.0006	13/16	888 ms	10.18M
MLPEDGE $h = 5$	Dot	Edge MLP	0.2854 ± 0.0005	14/16	471 ms	8.98M
MLPEDGE $h = 8$	Dot	Edge MLP	0.2862 ± 0.0010	12/16	536 ms	11.64M

Prompt masking. The most impactful single change was adding prompt-response masking ($y_{1:\ell-1} = -100$, where ℓ is the assistant prefix length): computing loss only on assistant tokens dropped KATPREY validation loss from 0.4202 to 0.2870 and eliminated role-marker leakage (the model generating “assistant” in responses). Table 2 shows the controlled comparison.

Table 2: Effect of prompt-response masking on KATPREY (8 000 steps).

Training objective	Val loss	Role-marker in responses
All tokens (no masking)	0.4202	Yes
Assistant tokens only	0.2870	No

MLPEDGE relative to B-spline edge-function variants. Within the B-spline/MLPEDGE subset, MLPEDGE remains the strongest edge-function replacement we measured. At $h = 5$, it improves mean validation loss over KANPREY grid=2 (0.2854 vs. 0.2883) and over KATPREY (0.2867),

while preserving visualizable 1D edge functions. This is narrower than a full KAN-family or MLP-replacement claim: BASISKAN Cheb $d = 3, g = 8$ and the MLP baseline are stronger on validation loss, and MLP remains much faster in the same-device inference sweep.

Table 4 still shows that $h = 1$ achieves the lowest single-seed MLPEDGE validation loss (0.2828), with loss increasing monotonically with h on that sweep. We therefore interpret $h \in \{1, 2\}$ as promising MLPEDGE settings, while using $h = 5$ and $h = 8$ for the cleaner controlled comparisons above.

5.2 Function-basis screen

The broader basis screen tests whether the edge-wise KAN topology can recover MLP-level quality with bases other than B-splines or per-edge MLPs. Table 3 summarizes the seed-42 2 000-step screen and the 8 000-step confirmation for promoted candidates. Only the rational square-denominator ablation and the Chebyshev degree-3 basis were promoted.

Table 3: GuppyLM function-basis screen. Pilot rows are seed-42, 2 000-step validation losses. Confirmation rows are 8 000-step mean $\pm$ std over seeds 42/43/44 when promoted.

Family	Config	2k pilot	8k confirmation	Decision
Rational	GR-KAN square denom., $P/Q = 5/4, g = 8$	0.3134	0.2780 ± 0.0009	confirmed
Polynomial	Chebyshev $d = 3, g = 8$	0.3151	0.2770 ± 0.0003	confirmed
Localized	inverse quadratic $c = 8, h = 1.5$	0.4169	—	reject
Gaussian	$c = 8, h = 1.0$	0.4198	—	reject
Gaussian	$c = 8, h = 1.5$	0.5123	—	reject
Gaussian	$c = 8, h = 2.0$	1.1174	—	reject
Hat/ReLU	ReLU-power $p = 2, c = 8$	0.3664	—	reject

The main positive result is the Chebyshev row: a recurrence-friendly polynomial basis matches the MLP validation-loss mean within the observed 3-seed noise, but it is slower in our unoptimized local implementation (550 ms vs. 309 ms for MLP in the seed-42 prompt sweep). This makes Chebyshev a plausible kernel-engineering target, not an immediate replacement for the MLP FFN. The rational square-denominator variant is safe but not a clear improvement over canonical GR-KAN.

The Gaussian/RBF branch fails for both quality and local kernel reasons. On shape 4096×384 with width scale 1.5, exact `torch.exp` was already faster than the tested approximations (2.86 ms forward / 7.02 ms backward) compared with `exp2` rewrite (3.32 ms / 8.08 ms), clamped `exp` (3.33 ms / 8.02 ms), and LUT interpolation (10.79 ms / 8.38 ms). The LUT was accurate (forward max error 7.7×10^{-6} ; gradient max error 3.0×10^{-9}), but slower on MPS. Training quality was also poor: the best Gaussian width was $h = 1.0$ at 0.4198 after 2 000 steps, far behind Chebyshev and rational variants.

5.3 Hidden-size ablation

Table 4: MLPEDGE hidden-size ablation (8 000 steps, seed 42). Inference latency measured on Apple M-series (MPS disabled; CPU for reproducibility). Bold = best val loss.

Hidden h	Val loss	Latency
1	0.2828	226 ms
2	0.2836	354 ms
5	0.2854	491 ms
10	0.2863	593 ms
20	0.2886	1262 ms

Figure 4 (Appendix) plots the efficiency-quality tradeoff curve.

5.4 Fair KANprey baseline

The original KANPREY result previously highlighted in Table 1 (val loss 0.4019) was trained for only 4 000 steps without prompt-response masking, making it an unfair comparison point. We therefore exclude it from the cleaned main table and retain it only as a protocol-correction example.

Under the corrected 8 000-step masked setup, the 3-seed KANPREY grid=2 baseline reaches $0.2883_{\pm 0.0005}$. This places it behind MLPEDGE $h = 5$ by 0.0029 validation-loss points and behind the same-protocol MLP baseline by 0.0113. The corrected B-spline baseline is still useful as the main controlled comparison for edge-function parameterizations, but it is no longer a strong headline model.

5.5 Scaling to GPT-2 size

To assess whether the small-scale KAN-family improvements of MLPEDGE generalize beyond the GUPPYLM testbed, we train a standard MLP transformer and an MLPEDGE transformer at GPT-2 small scale ($d_{\text{model}} = 768$, 12 heads, 12 layers) on the Wikitext-103 benchmark (Table 5).

Both models were trained for 20 000 steps with effective batch size 128 (gradient accumulation 4×32), cosine LR schedule (3×10^{-4} to 3×10^{-5} , 2 000 warmup steps), and AdamW with gradient checkpointing on a single RTX A6000 48 GB GPU. We report the best validation perplexity across all evaluation checkpoints.

Table 5: GPT-2 scale results on Wikitext-103 (20 000 steps, effective batch 128). Standard MLP FFN uses $\text{Linear}(768 \rightarrow 3072) \rightarrow \text{GELU} \rightarrow \text{Linear}(3072 \rightarrow 768)$ ($4\times$ expansion). MLPEDGE ($h = 8$, matched) is parameter-matched to the MLP baseline, isolating the effect of the per-edge factored structure from parameter count. Throughput measured end-to-end including gradient accumulation on RTX A6000.

Model	Val perplexity	Tokens/sec	Train time	Params
MLP transformer	16.58	54,820	13.3 h	124M
MLPEDGE ($h = 8$, matched)	20.96	48,921	14.9 h	125M

At GPT-2 scale, MLPEDGE ($h = 8$) achieves a validation perplexity of 20.96 versus 16.58 for the MLP baseline—a gap of 4.37 perplexity points (26%) in favour of the standard MLP. This does not reverse an MLP deficit at small scale; rather, it widens a gap that was already present on GUPPYLM, where MLP was also the strongest validation-loss baseline after cleanup. Throughput is 11% lower for MLPEDGE due to the einsum overhead of the per-edge factored computation at $d_{\text{model}} = 768$.

These results suggest that the per-edge factored structure provides an effective inductive bias at small scale but does not transfer cleanly to GPT-2 size with the same hidden-per-edge budget. Whether higher h or a different initialisation scheme would close this gap is left to future work. The GR-KAN experiment below tests the related question for a rational-basis variant.

5.6 GR-KAN at GPT-2 Scale

Setup. We train GR-KAN and a standard MLP transformer at the same scale as Section 5.5 ($d_{\text{model}} = 768$, 6 heads, 12 layers, $\sim 286\text{M}$ parameters) using the nanokan training harness¹ on the ClimbMix dataset (a curated web corpus with quality filtering, comparable to FineWeb-Edu). Both models are trained for 2 520 steps ($\sim 1.32\text{B}$ tokens, effective batch 256 sequences $\times$ 2 048 tokens) with the Muon optimizer [Jordan, 2024] on a single H100 80GB HBM3 GPU. The GR-KAN FFN uses grouped Pad  rational activations with polynomial degrees $P = 5$, $Q = 4$. We sweep the group count $g \in \{4, 8\}$ (group channel widths $D/g = 192$ and 96); a $g = 16$ run diverged to NaN and is excluded. The MLP baseline uses the standard $\text{Linear}(768 \rightarrow 3072) \rightarrow \text{SwiGLU} \rightarrow \text{Linear}(3072 \rightarrow 768)$ block. We report validation bits-per-byte (bpb) on a held-out ClimbMix split and DCLM CORE (22 ICL tasks, centered accuracy), neither of which is directly comparable to the Wikitext-103 perplexity in Table 5, but both isolate the FFN architecture difference under identical training conditions.

¹<https://github.com/ACS-USP/nanokan>

Implementation correction. After an earlier version of this work was circulated, a code review identified an error in the rational activation denominator. The original Triton kernels in `rational_kat_cu` computed:

$$Q(x) = 1 + |b_0| \cdot |x| + |b_1| \cdot |x|^2 + |b_2| \cdot |x|^3 + |b_3| \cdot |x|^4$$

applying absolute value per coefficient, rather than the canonical Safe Padé rational definition:

$$D(x) = b_0x + b_1x^2 + b_2x^3 + b_3x^4, \quad Q(x) = 1 + |D(x)|.$$

This bug was present from the original fork and affected all previously reported GPT-2 scale GR-KAN results. The forward and backward Triton kernels have been corrected; gradient consistency was verified via `torch.autograd.gradcheck` and a 10-step smoke test on H100 hardware. The results in Tables 6 and 7 reflect the canonical formula.

Engineering: fused Triton backward. The naïve PyTorch autograd backward for `GroupRational` dispatches ~ 480 independent CUDA kernels per training step. We use a single fused Triton backward kernel covering the Horner evaluation of $P(x)$, $D(x)$, $D'(x)$, elementwise ∂_x computation, and block-level coefficient gradient reductions in one launch. This achieves $\sim 290\text{K}$ tok/s throughput and $\sim 86\%$ MFU on H100 80GB HBM3 ($\sim 3\times$ improvement over the unfused baseline). The remaining throughput gap to MLP ($\sim 508\text{K}$ tok/s) reflects the arithmetic intensity of rational activations relative to dense cuBLAS matmuls.

Results. Tables 6 and 7 compare both corrected GR-KAN group counts against the MLP baseline on bpb and DCLM CORE. The best corrected configuration is $g = 4$: it reaches `val_bpb` = 0.890 versus 0.8464 for MLP (gap +0.044, $\sim 5.2\%$) and `CORE` = 0.128 versus 0.154 ($\Delta = -0.026$). Increasing the group count to $g = 8$ *hurts* both metrics (`val_bpb` = 0.9030, +0.057/ $\sim 6.7\%$; `CORE` = 0.116, $\Delta = -0.038$), so a higher group count provides no benefit at this scale. Against the stronger $g = 4$ variant, MLP leads on 18 of the 22 `CORE` tasks ($g = 4$ leads on `bigbench_cs_algorithms`, `agi_eval_lsar`, and `bigbench_dyck_languages`, with one tie); against $g = 8$, MLP leads on 19 of 22. The two consistent GR-KAN wins—`bigbench_cs_algorithms` (0.432/0.434 vs. 0.417) and `agi_eval_lsar` (0.114/0.109 vs. 0.011)—hold for both group counts, but the COPA advantage reported in the preliminary version does not survive the denominator correction (COPA is now an MLP win, 0.140 vs. 0.040/0.020).

Table 6: GR-KAN vs. MLP at GPT-2 scale: validation bpb (ClimbMix, $d_{\text{model}} = 768$, 12 layers, 286M parameters, 2 520 steps, H100 80GB HBM3, Muon optimizer). Canonical Safe Padé rational formula; both GR-KAN rows are trained from scratch with the corrected kernel.

Model	Val bpb ↓	Gap vs. MLP	Tok/sec
MLP transformer (SwiGLU)	0.8464	—	$\sim 508\text{K}$
GR-KAN ($g = 4$, $P/Q = 5/4$)	0.890	+0.044 (+5.2%)	$\sim 290\text{K}$
GR-KAN ($g = 8$, $P/Q = 5/4$)	0.9030	+0.057 (+6.7%)	$\sim 290\text{K}$

Interpretation. The corrected GR-KAN trails MLP at every tested group count—by +0.044 bpb ($g = 4$) to +0.057 bpb ($g = 8$), and by -0.026 to -0.038 on `CORE`. This is a larger gap than the $\sim 1.2\%$ incorrectly reported in a preliminary version, and the striking COPA advantage previously reported was an artifact of the denominator bug. The bpb gap is still smaller than the 26% gap for MLPEDGE, so rational activations remain a less damaging direction than per-edge MLPs at this scale, but the current result is negative relative to MLP. Notably, the group count does *not* behave as a free capacity knob: $g = 8$ is worse than $g = 4$ on both bpb and `CORE`, and a $g = 16$ run diverged to NaN, suggesting the group expansion needs reoptimization rather than naive scaling at this depth. Whether the gap closes with more training tokens, different initialization schemes for the rational coefficients, or a re-tuned group count is unknown. Each configuration here is a single run; we leave multi-seed replication, longer training horizons, and hyperparameter exploration to future work.

Table 7: GR-KAN ($g = 4, g = 8$) vs. MLP at GPT-2 scale: DCLM CORE benchmark (22 ICL tasks, centered accuracy unless noted; higher is better). Selected per-task breakdown; full 22-task CSVs are released with the code. Canonical formula, corrected kernel.

Task	GR-KAN ($g = 4$)	GR-KAN ($g = 8$)	MLP	Winner
CORE (aggregate)	0.128	0.116	0.154	MLP
HellaSwag (10-shot)	0.103	0.094	0.140	MLP
ARC-Easy	0.369	0.335	0.422	MLP
ARC-Challenge	0.009	−0.009	0.046	MLP
PIQA	0.313	0.312	0.357	MLP
LAMBADA (accuracy)	0.256	0.253	0.304	MLP
COPA	0.040	0.020	0.140	MLP
BoolQ	−0.081	−0.167	−0.058	MLP
bigbench_cs_algorithms	0.432	0.434	0.417	GR-KAN
agi_eval_lsar_ar	0.114	0.109	0.011	GR-KAN

6 Interpretability Analysis

6.1 Edge function reconstruction

For each KANLINEAR layer, we reconstruct all $d_{\text{in}} \times d_{\text{out}}$ learned 1D functions:

$$f_{i,j}(x) = W_{\text{base}}[j, i] \cdot \text{SiLU}(x) + \sum_k W_{\text{spline}}[j, i, k] \cdot B_{i,k}(x) \quad (6)$$

evaluating each on 200 points from its per-channel grid range.

MLP effective edge functions. For continuity with the earlier GuppyLM-style MLP comparison used in our interpretability notebook, we extract an “effective edge function” $f_{i,j}^{\text{MLP}}$ from a 2-layer FFN:

$$f_{i,j}^{\text{MLP}}(x) = \mathbf{W}_2[j, :] \cdot \text{ReLU}(\mathbf{W}_1[:, i] \cdot x + \mathbf{b}_1) + \mathbf{b}_2[j] \quad (7)$$

with $\mathbf{W}_1 \in \mathbb{R}^{768 \times 384}$ and $\mathbf{W}_2 \in \mathbb{R}^{384 \times 768}$. This is the scalar-valued contribution of input channel i to output channel j through the FFN, holding all other input channels at zero. We sample the same 200-point range $[-2, 2]$ for each pair.

6.2 Per-function metrics

We compute three scalar metrics for every edge:

Nonlinearity score (NLS): $\text{NLS} = \|f - f_{\text{lin}}\|_2 / (\|f\|_2 + \varepsilon)$, where f_{lin} is the best-fit affine function. $\text{NLS} \approx 0$ indicates a near-linear edge; $\text{NLS} \approx 1$ indicates strongly nonlinear behavior.

Activity: $\text{Act} = \|f\|_2 / \sqrt{n}$. Near-zero activity flags dead edges (prunable).

Roughness: $\text{Rough} = \text{mean}(|\Delta^2 f|)$, the mean absolute second finite difference.

Result summary. Across all 6 FFN layers and 884,736 edge functions: **87.8%** of edges have $\text{NLS} > 0.1$ (meaningfully nonlinear), and only **0.4%** are effectively dead (activity ≤ 0.01). NLS is remarkably consistent across layers (median 0.279–0.282 per layer), suggesting that nonlinearity is a stable property of KAN FFN layers rather than concentrated in specific depths. Figure 5 shows the NLS distributions for KAN vs. MLP effective edge functions; a Mann-Whitney U test gives $p < 10^{-300}$ for $\text{KAN} > \text{MLP}$, consistent across all 6 layers individually. This p -value is reported as a descriptive summary: edges within a layer share the same B-spline grid and basis functions, so they are not mutually independent, and the effective sample size is substantially smaller than 884,736. The directional conclusion—KAN edges are quantitatively more nonlinear than MLP edges at this scale—is supported by the per-layer consistency of the effect. MLP edges are also predominantly nonlinear (median $\text{NLS} = 0.268$ vs. 0.281 for KAN), suggesting that the difference is quantitative rather than qualitative.

6.3 Functional PCA

We apply truncated SVD to the $(d_{\text{out}} \cdot d_{\text{in}}) \times n_{\text{pts}}$ curve matrix per layer (curves normalised to a common $[0, 1]$ domain), giving functional principal components analogous to fPCA. Figure 6 (Appendix) shows the top-4 components for all 6 layers.

Result. The top 4 functional principal components explain **99.9%** of curve variance in every layer (PC1: 60–62%, PC2: 30–32%, PC3: 7%, PC4: <0.3%). We note that the finite B-spline basis bounds the edge-function space to a small number of degrees of freedom, so finding 99.9% variance in 4 components is partly a consequence of the parametrization itself. The substantive finding is *which* shapes dominate: fPC1 is a smooth monotone S-curve (sigmoid-like); fPC2–4 capture symmetric, asymmetric, and higher-frequency perturbations. The model converged to fewer functional archetypes than the available basis functions allow, indicating that training collapses the accessible function space to a small set of dominant motifs.

6.4 MLPEdge preserves interpretability

Figure 3 plots $f_{i,j}(x)$ for the same (i, j) pairs from KANPREY and MLPEDGE. Both produce smooth, visualizable 1D curves; the MLPEDGE curves are comparable in smoothness but have global support (SiLU-based), unlike the locally-supported B-spline functions.

6.5 Closed-form approximation of edge functions

A key interpretability promise of KANs is that learned edge functions can be matched to closed-form expressions. We test closed-form *approximability* by fitting a six-element library to the top-50 most active edges per layer: $\{\text{linear, quadratic, cubic, sigmoid, tanh, } x \cdot \text{SiLU}(x)\}$. For each edge, we report the best-fitting form by R^2 on 200 evaluation points within the training activation range. We do *not* claim this constitutes symbolic regression in the full sense: no search over expression trees is performed and the library is intentionally small.

Result. **93.3%** of the top-50 most active edges per layer admit a high-fidelity fit with $R^2 > 0.99$ (mean $R^2 = 0.9961$, median = 0.9993). The dominant form is cubic polynomial (91%), with swish ($x \cdot \text{SiLU}(x)$, 8%) and sigmoid (1%) accounting for the remainder. Figure 9 (Appendix) shows representative edge functions with their best-fit approximations. These results confirm that the most active edge functions are well-approximated by low-degree polynomials on the training domain, consistent with the KAN interpretability motivation. Crucially, however, high R^2 for a cubic fit is not discriminative: Taylor’s theorem guarantees that any smooth function on a bounded interval can be approximated arbitrarily well by a sufficiently high polynomial, so finding that cubic fits suffice does not identify the symbolic form. Furthermore, the six-element library is far too small to constitute a genuine symbolic search. A rigorous test would require a grammar-based search (e.g., PySR [Cranmer, 2023]) evaluated on held-out input ranges outside the training distribution. We regard the present analysis as characterizing the *complexity class* of the learned functions (smooth, bounded, polynomial-like) rather than identifying their symbolic forms.

6.6 Domain-selective circuits

We register forward hooks on the layer-2 KANLINEAR and record per-channel activations over 100 fish-domain prompts and 100 generic prompts sampled from the held-out test split. For each edge (i, j) we compute $\Delta_{i,j} = f_{i,j}(\bar{x}_i^{\text{fish}}) - f_{i,j}(\bar{x}_i^{\text{gen}})$, where $\bar{x}_i$ is the mean pre-KAN activation of input channel i averaged over all token positions and prompts in each domain. Figure 7 shows the resulting difference heatmap for layer 2.

Result. A sparse set of edges shows strong domain selectivity: the top-20 most selective edges have $|\Delta| > 0.05$ while the global mean is $|\Delta| \approx 0.002$. Input channel 147 has the highest mean $|\Delta|$ across all its output edges (mean 0.016 vs. layer mean 0.005) and appears in 3 of the top 5 most domain-selective pairs, identifying it as a domain-routing hub. This edge-level circuit structure is a correlational observation enabled by the KAN’s explicit 1D decomposition — an analysis that has no direct analogue in MLP FFNs, where activations are entangled across input dimensions.

6.7 Structured pruning

The interpretability analysis enables principled pruning: edges with low activity (small $\|f\|_2$) contribute negligibly to model output and can be zeroed without retraining. We evaluate two single-criterion pruning strategies: (a) *activity-only*: prune edge (i, j) if $\text{Act}_{i,j} < \tau_{\text{act}} \cdot \bar{\text{Act}}_\ell$ where $\bar{\text{Act}}_\ell$ is the layer mean; and (b) *NLS-only*: prune if $\text{NLS}_{i,j} < \tau_{\text{nls}}$.

Result. Activity-only pruning at $\tau_{\text{act}} = 0.3$ removes **3.6%** of edges with only $\Delta \text{val_loss} = +0.003$ zero-shot. After 3,000 steps of fine-tuning, the pruned model recovers to **0.297** val loss. Note: this is compared against the *uncorrected* KANPREY baseline (0.402, 4 000 steps, no masking); the corrected baseline is 0.2894, so the pruned+finetuned model (0.297) does *not* recover to baseline quality. This correctly frames the result: 3.6% sparsity incurs a permanent +0.008 val loss cost after fine-tuning. NLS-only pruning tells the complementary story: removing edges with $\text{NLS} < 0.05$ (just 2.8% of edges) causes an immediate +0.042 degradation, confirming that even the least-nonlinear KAN edges carry meaningful signal. Figure 8 (Appendix) plots the full quality–sparsity curves.

7 Discussion

7.1 Why MLPEdge works

The KAN topology—additive decomposition of each output feature over all input channels via independent 1D functions—remains the key architectural inductive bias inside the KAN-family design space. At small scale (384 hidden units, 60 K training samples), a tiny MLP per edge improves over the tested B-spline KAN variants even without a grid update mechanism, suggesting that the B-spline basis itself is not the only route to useful edge-wise functions.

7.2 Honest tradeoffs of MLPEdge

Advantages. No grid mechanism (no extra hyperparameters, no update step), and within the tested KAN-family variants, better validation loss and stronger heuristic prompt behavior than the B-spline baselines in Table 1.

Disadvantages. B-spline knots provide locally-supported basis functions, giving the spline path an inductive bias for smooth, locally-adapting functions; MLPEDGE uses SiLU, which has global support. Symbolic regression of learned edge functions is harder: B-splines can be approximated by simple polynomials per segment, while SiLU-based MLPs have no closed-form simplification. The connection to the Kolmogorov-Arnold theorem is equally valid: a SiLU-based MLP mapping $\mathbb{R} \rightarrow \mathbb{R}$ is a continuous univariate function and satisfies the theorem’s requirements. The practical distinction is that B-spline edges have a known, symbolic basis suited for regression, while SiLU-MLP edges have no closed-form simplification.

Scale caveat. The small-scale improvement of MLPEDGE over the tested B-spline KAN variants does not close the gap to the standard MLP baseline, and at GPT-2 size (Section 5.5) the gap widens substantially: the MLP baseline outperforms the parameter-matched MLPEDGE by 4.4 perplexity points while also running 11% faster. We therefore do not claim that MLPEDGE generalises beyond the GUPPYLM testbed.

7.3 GR-KAN at GPT-2 Scale

The corrected GR-KAN scale comparison in Section 5.6 is cautionary. Under the canonical Safe Padé formula and a group-count sweep, the best GR-KAN configuration ($g = 4$) reaches $\text{val_bpb} = 0.890$ versus 0.8464 for the MLP baseline at 286M parameters on ClimbMix—a 5.2% gap—while $g = 8$ is worse (0.9030, 6.7%). On DCLM CORE (22 ICL tasks), $g = 4$ scores 0.128 and $g = 8$ scores 0.116 versus 0.154 for MLP ($\Delta = -0.026$ and -0.038), with MLP leading on 18 of 22 tasks against $g = 4$ and no COPA advantage at either group count. The bpb gap is smaller than the 26% MLPEDGE gap at GPT-2-small scale, so rational activations remain a plausible but unproven route for future work. These results do not establish that GR-KAN is competitive with MLP, nor that group-shared Padé activations scale cleanly.

Because all smaller pre-fix GR-KAN runs used the wrong denominator formula, we do not draw conclusions from their reported validation loss, modular-staging behavior, or external comparisons. Those rows require fresh denominator-corrected reruns before they can re-enter the paper.

7.4 Relation to PowerMLP

Tang et al. [2025] show algebraic equivalence between B-splines and ReLU-power compositions, but do not train or evaluate per-edge MLP functions in a language model. MLPEDGE is independently motivated and evaluated here; the equivalence result provides a theoretical connection but does not anticipate our empirical findings.

8 Limitations

GUPPYLM is a controlled testbed, not a production-scale model. Our GPT-2 scale experiment (Section 5.5) shows that MLPEDGE’s advantage does *not* transfer to $d_{\text{model}} = 768$: the per-edge factored structure falls 26% behind the MLP baseline in perplexity at matched parameter count. Whether a larger per-edge hidden size or different initialisation would recover competitive performance for MLPEDGE at scale remains open. Section 5.6 shows that canonical GR-KAN with rational activations also trails MLP at 286M parameters at every tested group count: +0.044 ($g = 4$) to +0.057 ($g = 8$) bpb, -0.026 to -0.038 DCLM CORE, and lower throughput. Although this bpb gap is smaller than the MLPEDGE gap, it is still a negative result relative to the MLP baseline and cannot support a GR-KAN architecture claim. The current corrected comparison is a single run per group count ($g \in \{4, 8\}$); additional seeds, other initialization schemes, and longer training horizons remain open. The keyword-match evaluation metric on GUPPYLM is coarse; the GPT-2 scale experiment uses bpb (ClimbMix) and DCLM CORE (ICL benchmark). Interpretability results apply to B-spline KANLinear; MLPEDGE functions are visualizable but harder to analyze symbolically. The domain-selective circuit analysis (Section 6.6) is correlational. We tested causal relevance by scaling the spline output of the top-5 most domain-selective edges by $\alpha \in \{0, 0.5, 2, 5\}$ (zeroing at $\alpha = 0$, amplifying at $\alpha = 5$) and measuring the resulting change in validation loss on 100 fish-domain vs. 100 generic prompts. The maximum observed asymmetry ($|\Delta\text{loss}_{\text{fish}} - \Delta\text{loss}_{\text{generic}}|$) across all α values is < 0.004 — comparable to noise and far below what is needed to claim selective behavioral impact. This null result likely reflects redundancy across the 384 input channels and 6 transformer layers; single-edge interventions are insufficient. Establishing causal circuit roles would require larger multi-edge ablations or activation patching across the full residual stream. Training logs for early experiments were not saved step-by-step; Figure 2 uses logged val_loss from re-runs with the corrected training loop.

9 Related Work

KAN and variants. Liu et al. [2025] introduce KANs and demonstrate symbolic regression on scientific functions. Yang and Wang [2025] extend KANs to attention layers. Liu et al. [2024] (KAN 2.0) add MultKAN and grid extension techniques. Bodner et al. [2024] and others apply KANs to vision tasks.

Efficient KAN. Blealtan [2024] implement KANLINEAR efficiently using pre-computed B-spline grids; we build on this implementation.

Additive models and interpretability. Neural additive models [Agarwal et al., 2021] represent each feature by a learned shape function, sharing the “one function per input dimension” interpretability goal. KANs extend this to the edge level, yielding a richer family of decompositions.

PowerMLP. Tang et al. [2025] prove algebraic equivalence between KAN spline edges and ReLU-power compositions, providing theoretical motivation for replacing splines with polynomial networks. MLPEDGE pursues a similar substitution but uses learned per-edge MLPs and evaluates the approach in a language model setting.

10 Conclusion

We presented the first empirical interpretability study of KAN edge functions in a language model, finding that 87.8% of 884,736 edges are meaningfully nonlinear ($\text{NLS} > 0.1$) and only 0.4% are dead. At GUPPYLM scale, the function-basis screen finds that a grouped Chebyshev degree-3 FFN reaches 0.2770 ± 0.0003 validation loss over three seeds, matching the same-protocol MLP mean (0.2770 ± 0.0008) but running slower in the seed-42 inference sweep (550 ms vs. 309 ms). Corrected canonical GR-KAN remains close (0.2780 ± 0.0005), and a square-denominator rational ablation is similar (0.2780 ± 0.0009), but no tested KAN-family FFN dominates MLP on both quality and latency. MLPEDGE remains the strongest visualizable edge-function replacement among the B-spline and MLPEDGE rows: it reaches 0.2854 ± 0.0005 validation loss at $h = 5$, compared with 0.2883 ± 0.0005 for the standard-attention B-spline baseline and 0.2867 ± 0.0006 for KATPREY. At GPT-2 scale (124M parameters, Wikitext-103), the parameter-matched MLPEDGE ($h = 8$) trails the MLP baseline by a substantial margin (perplexity 20.96 vs. 16.58), indicating that the per-edge factored structure does not scale straightforwardly beyond the small-model regime explored here.

All pre-fix GR-KAN runs outside the corrected GUPPYLM rerun are excluded from the evidence base because they used the wrong rational denominator formula. The removed modular-staging rows still require fresh denominator-corrected reruns; the old d12 group-count rows, however, have been replaced by the corrected from-scratch runs reported next. We additionally scale GR-KAN to 286M parameters on ClimbMix (Section 5.6), finding that the best corrected configuration ($g = 4$) reaches $\text{val_bpb} = 0.890$ versus 0.8464 for the MLP baseline—a 5.2% gap, widening to 6.7% at $g = 8$ —after engineering a fused Triton backward kernel for the group-rational activation ($\sim 3\times$ faster than the unfused rational baseline; $\sim 290\text{K}$ tok/s versus $\sim 508\text{K}$ tok/s for MLP). On DCLM CORE (22 ICL tasks), $g = 4$ scores 0.128 and $g = 8$ scores 0.116 versus 0.154 for MLP ($\Delta = -0.026$ and -0.038), with MLP leading on 18 of 22 tasks and no COPA advantage under the corrected denominator. Thus, the corrected GR-KAN result is a useful negative/cautionary scale result: rational activations appear less damaging than the per-edge MLPEDGE factorization at this scale, but they do not yet match the MLP baseline.

The GUPPYLM testbed and all model code are publicly available: <https://github.com/HCAI-USP/kan-guppylm>.

Acknowledgments and Disclosure of Funding

This work was carried out at the Human-Centered Artificial Intelligence group (HCAI) of the University of São Paulo. We thank the GuppyLM dataset contributors and the EfficientKAN open-source implementation. Small-scale experiments were run on a personal Apple Silicon workstation. The Wikitext-103 GPT-2 scale experiments used an NVIDIA RTX A6000 GPU via RunPod community cloud; the ClimbMix GR-KAN scale comparison used an H100 80GB HBM3 GPU. No external compute grants were used.

References

- Rishabh Agarwal, Levi Melnick, Nicholas Frosst, Xuezhou Zhang, Ben Lengerich, Rich Caruana, and Geoffrey Hinton. Neural additive models: Interpretable machine learning with neural nets. In *Advances in Neural Information Processing Systems*, 2021. URL <https://arxiv.org/abs/2004.13912>.
- arman bd. GuppyLM-60K-Generic: A fish-personality chatbot dataset. <https://huggingface.co/datasets/arman-bd/guppylm-60k-generic>, 2024.
- Blealtan. EfficientKAN: Efficient implementation of KAN. <https://github.com/Blealtan/efficient-kan>, 2024.
- Alexander Dylan Bodner, Antonio Santiago Tepsich, Jack Natan Spolski, and Santiago Pourteau. Convolutional KANs: When Kolmogorov-Arnold networks meet Diffusion. <https://arxiv.org/abs/2406.13155>, 2024.
- Miles Cranmer. Interpretable machine learning for science with PySR and SymbolicRegression.jl. *arXiv preprint arXiv:2305.01582*, 2023.

Keller Jordan. Muon: An optimizer for hidden layers in neural networks. <https://github.com/KellerJordan/modded-nanogpt>, 2024.

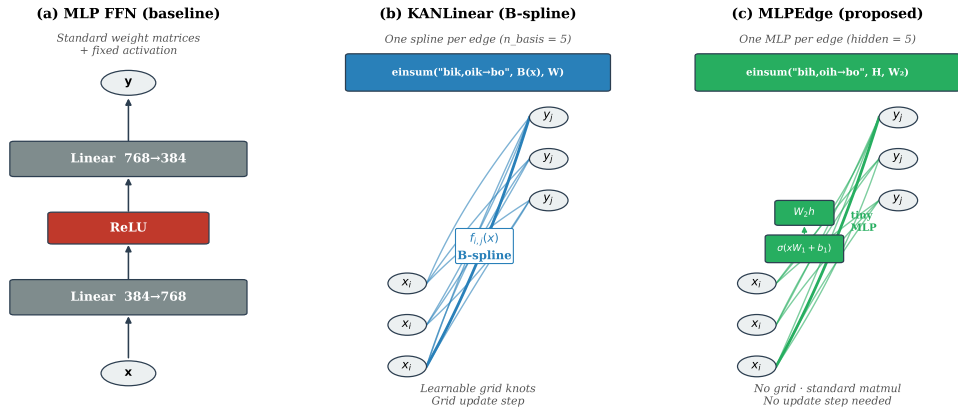
Ziming Liu, Pingchuan Zhu, Ivan Fernandez-Corbaton, and Max Tegmark. KAN 2.0: Kolmogorov-Arnold networks meet science. *arXiv preprint arXiv:2408.10205*, 2024.

Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y Hou, and Max Tegmark. KAN: Kolmogorov-Arnold Networks. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2404.19756>.

Ruichen Tang, Fenglei Wu, Qingyun Xu, Junhao Liao, Hao Lu, and Li Cheng. PowerMLP: An efficient version of KAN. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025. URL <https://arxiv.org/abs/2412.13571>.

Xingyi Yang and Xinchao Wang. KAT: Kolmogorov-Arnold Transformer. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2409.10594>.

A Architecture Diagram



KAN topology (additive decomposition over edges) is preserved in all three variants.

Figure 1: **Three FFN variants.** *Left:* Standard MLP FFN (two linear layers + ReLU). *Center:* KANLinear — each edge $f_{i,j}$ is a B-spline function with learnable coefficients and adaptive knots. *Right:* MLPEdge — each edge $f_{i,j}$ is a tiny MLP ($\mathbb{R} \rightarrow \mathbb{R}$), realized via the same einsum structure.

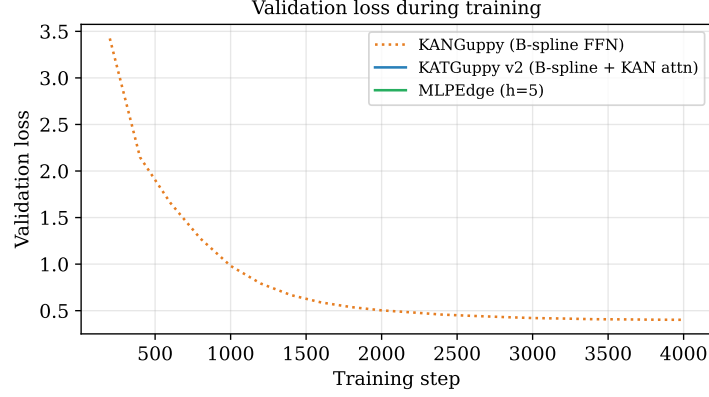


Figure 2: Validation loss vs. training step for all four models. The prompt-masking fix (dashed vertical line at step 0 of KAT v2 and MLPEGE runs) accounts for the discontinuous improvement relative to the initial KANPREY run.

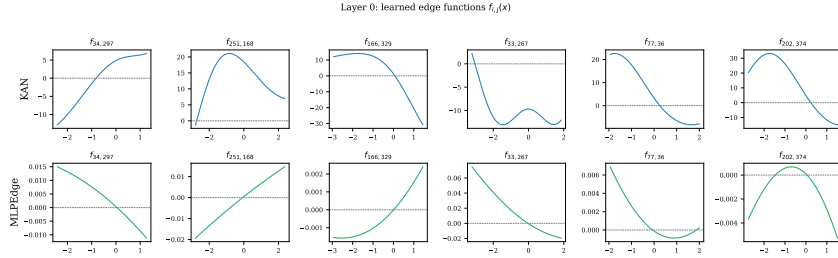


Figure 3: Learned 1D edge functions $f_{i,j}(x)$ for the same (i, j) pairs from layer 0 of KANPREY (B-spline, *left*) and MLPEGE (*right*). Both produce smooth, visualizable curves; MLPEGE functions have global support while B-spline functions have locally-supported knots (vertical tick marks, bottom).

B Training Loss Curves

C Spline vs. MLPEGE Edge Visualization

D MLPEGE Hidden-Size Ablation

E NLS Distribution and MLP Comparison

F Functional PCA Components

G Domain-Selective Circuit Heatmap

H Structured Pruning Quality–Sparsity Curves

I Closed-Form Approximation of Edge Functions

J Reproducibility Checklist

This section documents the exact software, hardware, datasets, commands, and seeds needed to reproduce the main experiments.

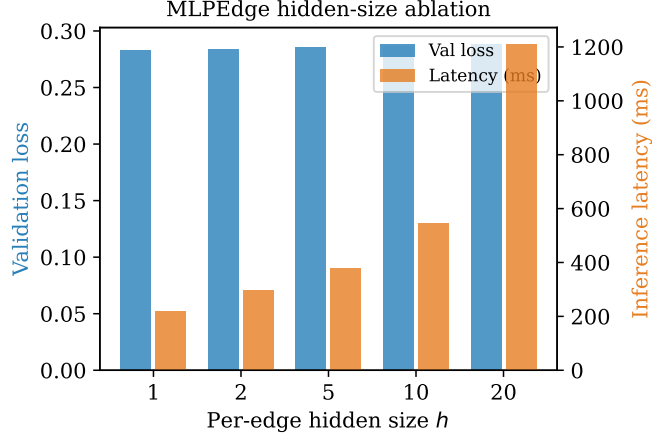


Figure 4: MLPEDGE hidden-size ablation. Left axis: validation loss; right axis: inference latency. $h = 1$ achieves the best validation loss (0.2828); $h = 2$ is the practical sweet spot (0.2836, 354 ms). We use $h = 5$ as our main comparison point for parameter-count matching with $n_{\text{basis}} = 5$ B-spline models.

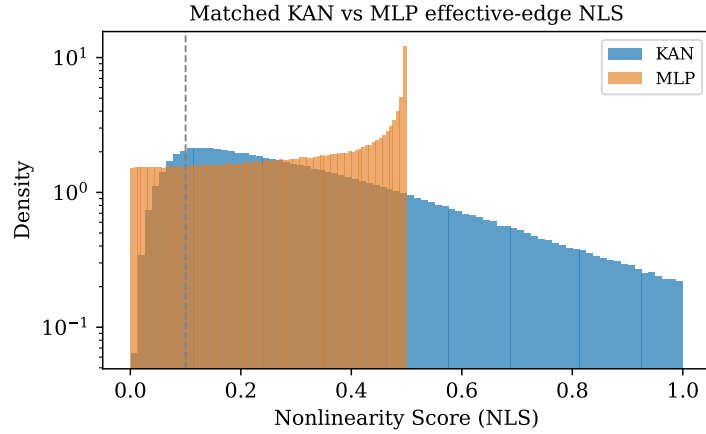


Figure 5: Overlaid histograms of nonlinearity score (NLS) for KAN edge functions (884,736 edges across 6 FFN layers, blue) and MLP effective edge functions (orange). Log-scale y-axis. Mann-Whitney U test: $p < 10^{-300}$ (KAN > MLP). Both distributions peak well above 0.1, with KAN edges slightly more nonlinear (median 0.281 vs. 0.268).

J.1 Code and Revisions

- **kan-guppylm** (GuppyLM runs, MLPEDGE GPT-2 Wikitext-103): <https://github.com/HCAI-USP/kan-guppylm>, commit 3c8ea92 (2026-06-01).
- **nanokan** (GR-KAN ClimbMix d12 runs): <https://github.com/ACS-USP/nanokan>, commits 8b21df2 ($g = 4$ run) and 0e0bea5 ($g = 8$ run); the canonical Safe Padé denominator is provided by the pinned `rational_kat_cu` kernel below.
- **nanochat** (training harness dependency): <https://github.com/karpathy/nanochat>, commit dc54a1a (2026-05-22).
- **rational_kat_cu** (rational activation kernels): https://github.com/Adamdad/rational_kat_cu, commit 41a20b5, with the corrected Safe Padé denominator and fused Triton backward kernels.

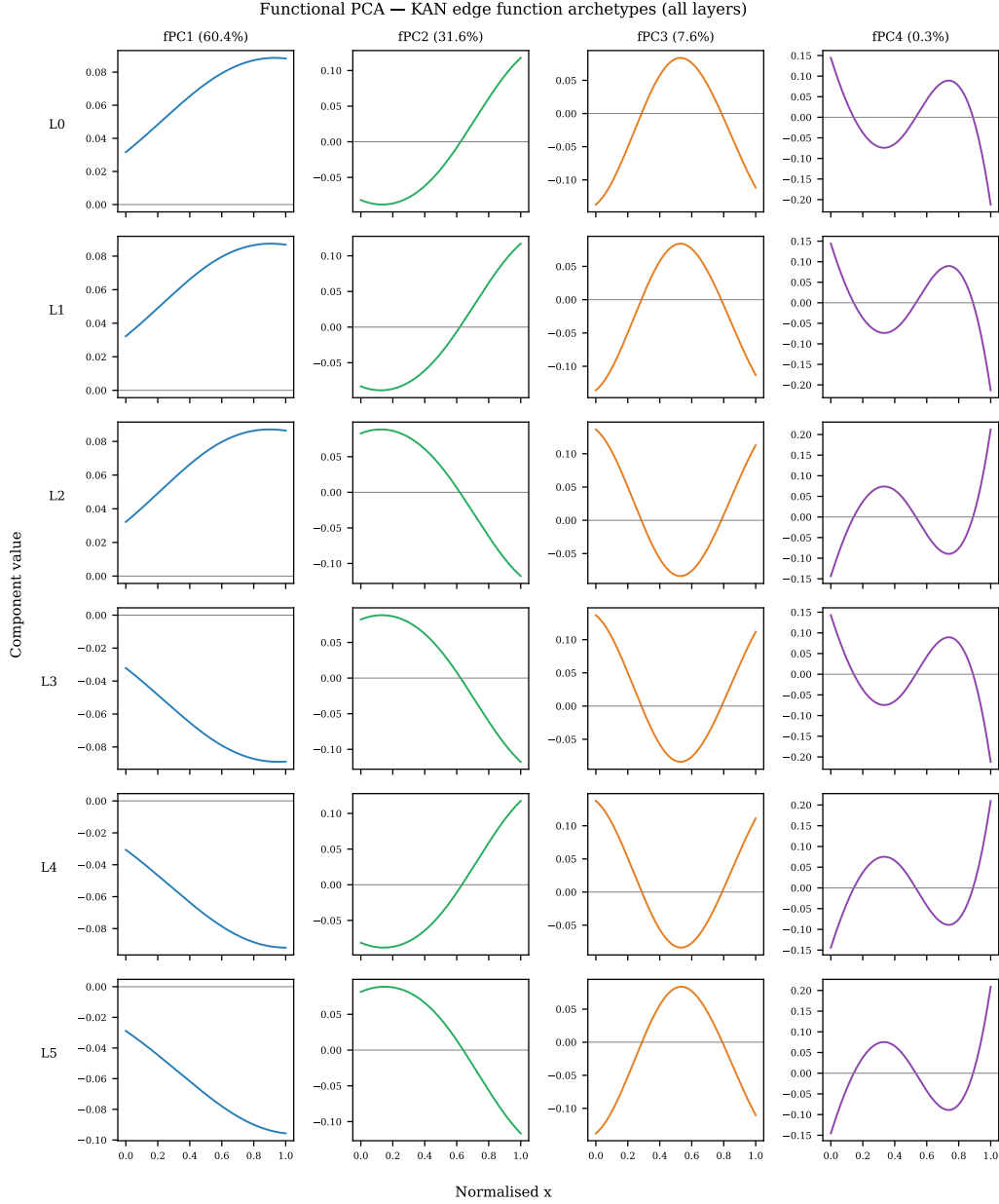


Figure 6: Top-4 functional principal components for all 6 KAN FFN layers of KANPREY. PC1–4 together explain **99.9%** of curve variance in every layer (PC1 alone: 60–62%). Note: the finite B-spline basis bounds the edge-function space to a small number of degrees of freedom; the 4-PC result reflects that training further collapses this space to dominant archetypes (see Section 6).

J.2 Rational Denominator Correction

All pre-correction GR-KAN d12 checkpoints are excluded from the publication evidence except as historical invalidation context. The bug and fix are as follows:

- **Canonical (correct):** $Q(x) = 1 + |b_0x + b_1x^2 + b_2x^3 + b_3x^4|$.
- **Retracted (bugged):** $Q(x) = 1 + |b_0||x| + |b_1||x|^2 + |b_2||x|^3 + |b_3||x|^4$.

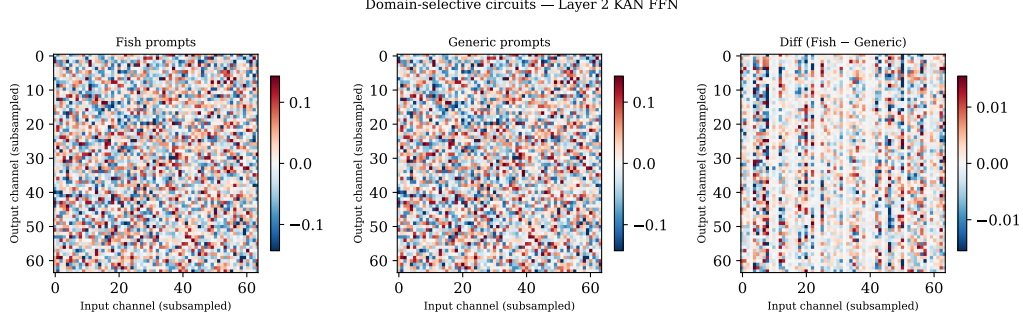


Figure 7: Per-edge output difference $\Delta_{i,j} = f_{i,j}(\bar{x}_i^{\text{fish}}) - f_{i,j}(\bar{x}_i^{\text{gen}})$ for layer 2 of KANPREY. *Left*: mean edge outputs for fish-domain prompts. *Centre*: mean edge outputs for generic prompts. *Right*: difference (fish – generic). Red = fish-selective, blue = generic-selective. A sparse set of edges ($|\Delta| > 0.05$) shows strong domain selectivity; input channel 147 appears in 3 of the 5 most domain-selective pairs.

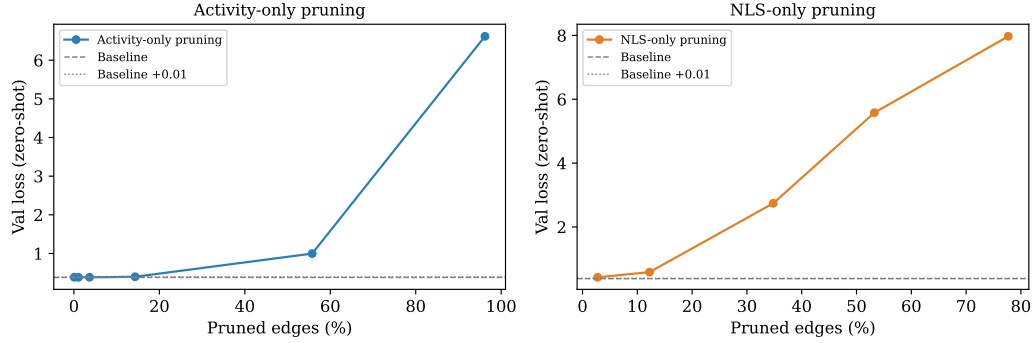


Figure 8: Zero-shot val loss vs. fraction of edges pruned for activity-only (*left*) and NLS-only (*right*) pruning strategies. Activity-only pruning removes 3.6% of edges at +0.003 val loss; NLS-only pruning causes steep degradation even at low sparsity, confirming that nonlinear edges are functionally load-bearing. Dashed line: baseline; dotted line: baseline + 0.01.

Gradient consistency of the corrected kernel was verified via `torch.autograd.gradcheck` on H100 hardware (max errors: $dx = 3.81 \times 10^{-6}$, $da = 8.58 \times 10^{-6}$, $db = 9.54 \times 10^{-6}$, float32).

J.3 Environment Versions

Table 8: Software environment for each experiment family.

	GuppyLM Track A	MLPEdge GPT-2	GR-KAN ClimMix d12
OS	Darwin 25.4.0 arm64	Ubuntu 22.04	Ubuntu 22.04
Python	3.13.5	3.10.12	3.10.12
PyTorch	2.12.0 (MPS)	2.4.1 (CUDA 12.4)	2.5.1 (CUDA 12.4)
Triton	—	—	3.1.0
Flash Attn	—	—	FA3 (Hopper SM90)
Muon	—	—	Newton-Schulz-5

J.4 Hardware

- **GuppyLM Track A**: Apple M4 Pro, 14 logical cores, 25.77 GB unified memory, MPS backend. Training wall-clock: ~ 10 min per 8,000-step run.

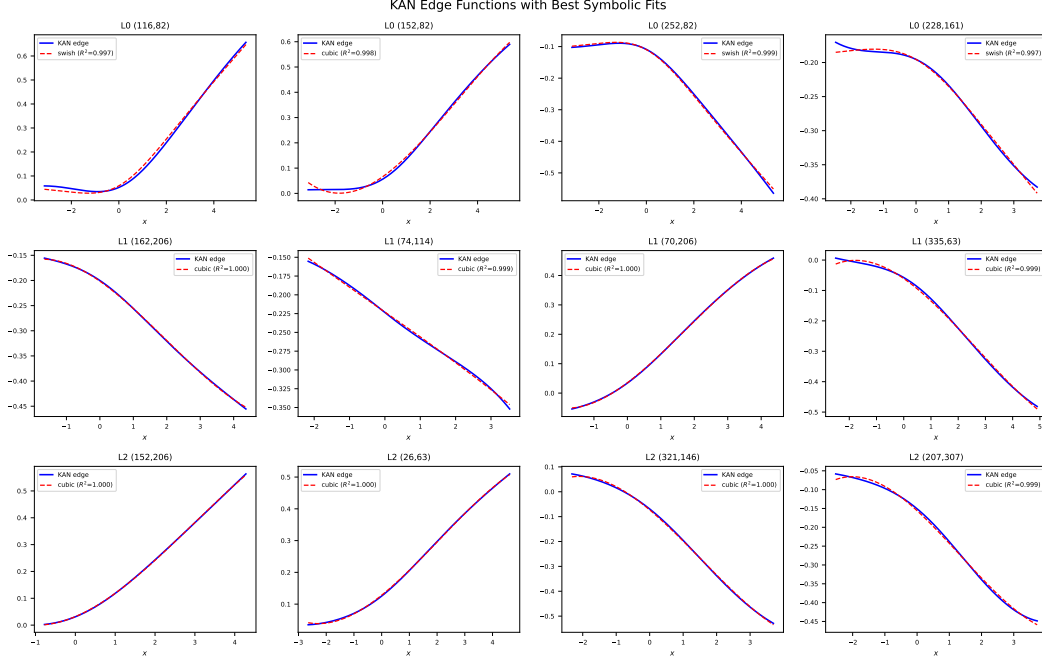


Figure 9: Representative KAN edge functions (blue) from layer 0 of KANPREY with their best-fit closed-form approximations (orange dashed). Candidate forms: linear, quadratic, cubic, sigmoid, tanh, $x \cdot \text{SiLU}(x)$. R^2 is reported for each fit within the training activation range. 93.3% of the top-50 most active edges per layer achieve $R^2 > 0.99$ (mean $R^2 = 0.9961$); the dominant form is cubic polynomial (91%). Note: high R^2 for a cubic on a bounded interval reflects the smoothness of the learned functions, not a definitive symbolic identification (see Section 6.5).

- **MLPEdge GPT-2 Wikitext-103**: single NVIDIA RTX A6000 48 GB (RunPod community cloud). Training wall-clock: 13.3 h (MLP), 14.9 h (MLPEdge).
- **GR-KAN ClimMix d12**: single NVIDIA H100 80 GB HBM3 (RunPod community cloud). Training wall-clock: ~ 2.0 h per run (2,520 steps); step time ~ 1.76 s, throughput ~ 290 K tok/s, $\sim 88\%$ BF16 MFU (steady state, from the run logs). Eval hardware: NVIDIA RTX PRO 4500 Blackwell 32 GB (separate pod).

J.5 Dataset Provenance and Tokenizers

- **GuppyLM**: arman-bd/guppylm-60k-generic (HuggingFace; MIT license, DOI 10.57967/hf/8339). Tokenizer: tokenizer.json (GPT-2 BPE, vocab 2,393). Max sequence length: 128 tokens. Objective: assistant-token-only loss; prompt token targets set to -100 .
- **MLPEdge GPT-2 Wikitext-103**: Salesforce/wikitext, configuration wikitext-103-v1 (Merity et al., 2017; CC BY-SA 3.0 / GFDL). GPT-2 BPE tokenizer (vocab 50,257).
- **GR-KAN ClimMix d12**: nanochat’s ClimMix corpus, traced by nanochat to NVIDIA Nemotron-ClimbMix (400B tokens; CC BY-NC 4.0; research and development use). RustBPE tokenizer trained on 30 ClimbMix shards, `-max-chars=2B -vocab-size=32768`. Tokenizer artifacts: tokenizer.pkl + token_bytes.pt. **Evaluation guard**: the eval pod uses the exact same tokenizer artifact from training (30-shard build), not a reduced-shard rebuild.

J.6 GuppyLM Track A Commands and Seeds

All cleanup runs: 8,000 steps, batch size 32, kanprey.train defaults (AdamW, cosine LR, 200 warmup steps). Seed list: {42, 43, 44}.

```

# Same-protocol MLP baseline
uv run python -m kanprey.train --model mlp --steps 8000 \
  --batch-size 32 --checkpoint-dir checkpoints/mlp_s42 --seed 42
# ... repeated for seeds 43, 44

# B-spline KAN grid=2
uv run python -m kanprey.train --model kan --grid-size 2 --steps 8000 \
  --batch-size 32 --checkpoint-dir checkpoints/kan_grid2_s42 --seed 42
# ... repeated for seeds 43, 44

# Parameter-matched MLPEdge h=8
uv run python -m kanprey.train --model mlpedge --mlp-edge-hidden 8 \
  --steps 8000 --batch-size 32 \
  --checkpoint-dir checkpoints/mlpedge_h8_s42 --seed 42
# ... repeated for seeds 43, 44

# Corrected canonical GR-KAN
uv run python -m kanprey.train --model grkan --steps 8000 \
  --batch-size 32 --checkpoint-dir checkpoints/grkan_corrected_s42 \
  --seed 42
# ... repeated for seeds 43, 44

Existing KAT and MLPEdge  $h = 5$  rows use the same seeds from checkpoints/kat_s* and
checkpoints/mlpedge_s*.

```

J.7 GuppyLM Function-Basis Screen

The function-basis screen first ran the Gaussian/RBF microbenchmark gate:

```

uv run python scripts/benchmark_basis_kernels.py \
  --shape 4096x384 --device auto --dtype float32 \
  --centers 8 --width-scale 1.5 --repeats 20 \
  --output results/basis_gaussian_microbench_4096x384_h15.json

```

The Stage-1 pilot matrix used 2000 steps, batch size 32, seed 42, and checkpoints/basis_pilots/<name> output directories. The promoted Stage-2 confirmation rows used:

```

# Square-denominator GR-KAN confirmation
uv run python -m kanprey.train --model grkan --steps 8000 \
  --batch-size 32 --seed 42 --grkan-denominator square \
  --grkan-m 5 --grkan-n 4 --grkan-groups 8 \
  --checkpoint-dir checkpoints/basis_confirm/grkan_square_p5q4_g8_s42
# ... repeated for seeds 43, 44

# Chebyshev degree-3 grouped basis confirmation
uv run python -m kanprey.train --model basis --steps 8000 \
  --batch-size 32 --seed 42 --basis-family chebyshev \
  --basis-degree 3 --basis-groups 8 --basis-input-norm tanh \
  --checkpoint-dir checkpoints/basis_confirm/cheb_d3_g8_s42
# ... repeated for seeds 43, 44

```

Primary artifacts are results/basis_stage1_summary.json, results/basis_stage2_summary.json, results/basis_stage2_run_manifest.json, results/basis_stage2_eval.json, and results/basis_stage2_diagnostics.json.

J.8 GuppyLM Unified Evaluation

```

uv run python eval_cases.py --json --max-tokens 64 \
  --run MLP=checkpoints/mlp_s42/best.pt \

```

```

--run KAN=checkpoints/kan_grid2_s42/best.pt \
--run KAT=checkpoints/kat_s42/best.pt \
--run MLPEdgeH5=checkpoints/mlpedge_s42/best.pt \
--run MLPEdgeH8=checkpoints/mlpedge_h8_s42/best.pt \
--run GRKAN=checkpoints/grkan_corrected_s42/best.pt \
--run GRKANSquare=checkpoints/basis_confirm/grkan_square_p5q4_g8_s42/best.pt \
--run ChebD3=checkpoints/basis_confirm/cheb_d3_g8_s42/best.pt

```

Settings: device mps, tokenizer tokenizer.json, temperature 0.7, top- k 50, max new tokens 64, 16 prompts. The 16-prompt score is a pass/fail heuristic on manual criteria; it is not a benchmark and should be interpreted with binomial uncertainty. Latency: mean and median across 16 prompts $\times$ 64 generated tokens.

J.9 MLPEdge GPT-2 Wikitext-103

- Model config: $d_{\text{model}} = 768$, 12 heads, 12 layers, context 1,024.
- MLP FFN: $\text{Linear}(768 \rightarrow 3072) \rightarrow \text{GELU} \rightarrow \text{Linear}(3072 \rightarrow 768)$ ($4 \times$ expansion).
- MLPEDGE $h = 8$: per-edge MLP hidden size 8, parameter-matched to MLP.
- Training: 20,000 steps, effective batch 128 (gradient accumulation 4×32), cosine LR $3 \times 10^{-4} \rightarrow 3 \times 10^{-5}$, 2,000 warmup, AdamW, gradient checkpointing enabled.
- Reported: best validation perplexity across all eval checkpoints.

J.10 GR-KAN ClimbMix d12

- Harness: nanokan wrapping nanochat with -ffn-type switch.
- Model tags: d12-mlp, d12-grkan-g4, d12-grkan-g8.
- Config: $d_{\text{model}} = 768$, 6 heads, 12 layers, sequence length 2,048, $\sim 286\text{M}$ parameters.
- GR-KAN: group counts $g \in \{4, 8\}$ (best: $g = 4$; $g = 16$ diverged), $P = 5$, $Q = 4$, canonical Safe Padé denominator. Rational init: identity (rat1), swish (rat2).
- MLP baseline: $\text{Linear}(768 \rightarrow 3072) \rightarrow \text{SwiGLU} \rightarrow \text{Linear}(3072 \rightarrow 768)$.
- Training: 2,520 steps, effective batch $256 \text{ seq} \times 2,048 \text{ tok} = 524,288 \text{ tok/step}$, $\sim 1.32\text{B}$ total tokens. Optimizer: Muon (Newton-Schulz-5) for 2D weights, AdamW for scalars and rational coefficients. BF16 mixed-precision, Flash Attention 3.
- Eval: BPB and DCLM CORE (22 ICL tasks, centered accuracy). Full per-task CSV available at <https://github.com/ACS-USP/nanokan>.
- RunPod commands:

```

python3 scripts/runpod_launch.py test
python3 scripts/runpod_launch.py train --model grkan \
  --volume-id <volume_id> --steps 2520
python3 scripts/runpod_launch.py eval \
  --model-tags d12-grkan-g4 d12-grkan-g8 d12-mlp --secure

```

J.11 Checkpoint Lineage

J.12 Statistical Reporting

- GuppyLM validation loss: mean $\pm$ std over 3 seeds for each model row.
- GuppyLM 16-prompt heuristic: pass/fail score; not a benchmark. Binomial standard error $\sim \pm 2.5$ prompts at $n = 16$. Differences of 1–2 correct responses are not statistically meaningful.
- MLPEdge GPT-2 Wikitext-103: single-run comparison (no multi-seed replication yet).
- GR-KAN ClimbMix d12: single corrected run per group count ($g = 4$, $g = 8$). DCLM CORE per-task results are reported with the aggregate; per-task advantage claims would require confidence intervals and predeclared analysis.

Table 9: Checkpoint paths for every result row.

Result row	Checkpoints
GuppyLM MLP	mlp_s42, mlp_s43, mlp_s44
GuppyLM KAN grid=2	kan_grid2_s42, kan_grid2_s43, kan_grid2_s44
GuppyLM KAT grid2+KAT	kat_s42, kat_s43, kat_s44
GuppyLM MLPEdge $h = 5$	mlpedge_s42, mlpedge_s43, mlpedge_s44
GuppyLM MLPEdge $h = 8$	mlpedge_h8_s42, mlpedge_h8_s43, mlpedge_h8_s44
GuppyLM corrected GR-KAN	grkan_corrected_s42, grkan_corrected_s43, grkan_corrected_s44
GuppyLM GR-KAN square denom.	basis_confirm/grkan_square_p5q4_g8_s42, basis_confirm/grkan_square_p5q4_g8_s43, basis_confirm/grkan_square_p5q4_g8_s44
GuppyLM BASISKAN Cheb $d = 3, g = 8$	basis_confirm/cheb_d3_g8_s42, basis_confirm/cheb_d3_g8_s43, basis_confirm/cheb_d3_g8_s44
MLPEdge GPT-2 MLP	checkpoints/scale/mlp/best.pt
MLPEdge GPT-2 $h = 8$	checkpoints/scale/mlpedge_h8/best.pt
GR-KAN d12 ($g = 4$)	checkpoints/nanochat/b-g4-20260610T193454Z-full-d12-grkan-g4/d12-grkan-g4/model_002520.pt plus meta_002520.json, tokenizer.pkl, and token_bytes.pt
GR-KAN d12 ($g = 8$)	checkpoints/nanochat/20260608T193716Z-d12-grkan-g8-corrected-r2/d12-grkan-g8-corrected-r2/model_002520.pt plus matching metadata and tokenizer
d12 MLP	checkpoints/nanochat/d12-mlp/model_002520.pt plus matching tokenizer and eval artifacts

J.13 Known Limitations Preserved in the Manuscript

1. No tested KAN-family GuppyLM model dominates MLP on both validation loss and latency; Chebyshev $d = 3, g = 8$ matches the MLP validation-loss mean but is slower.
2. Corrected GuppyLM GR-KAN is close to MLP but not a replacement claim; the square-denominator rational ablation is also close but not a quality/latency win.
3. Parameter-matched MLPEdge $h = 8$ does not close the MLP gap at GuppyLM scale.
4. MLPEdge $h = 8$ fails to scale cleanly to GPT-2-small on Wikitext-103 (26% perplexity gap).
5. Corrected canonical GR-KAN trails MLP at 286M parameters on BPB, DCLM CORE, and throughput at every tested group count ($g = 4$ best, $g = 8$ worse, $g = 16$ diverged).
6. Pre-fix GR-KAN checkpoints outside the corrected GuppyLM rerun remain excluded from the evidence base until corrected reruns replace them.
7. Domain-selective edge analysis is correlational; causal circuit claims are not supported.